

QuickAPI

高性能 API 框架

PHP 7.4+ | Medoo ORM | inhere/sroute

中文用户手册 v1.0

目录

1. 简介
 2. 核心特性
 3. 快速开始
 4. 数据库操作
 5. 缓存系统
 6. 路由系统
 7. 请求与响应
 8. 中间件
 9. 验证器
 10. 性能优化
 11. 配置参考
- 附录 A 目录结构
- 附录 B 许可证

1. 简介

QuickAPI 是一个基于 PHP 7.4+ 的高性能 API 框架，结合了 Medoo ORM 的强大功能和类 ThinkPHP5 的简洁链式调用语法，同时融合 inhere/sroute 的极速路由匹配。框架默认关闭 PHP Session，实现无状态设计，最大化运行性能。

1.1 主要特点

极简数据库操作 - 类似 ThinkPHP5 的链式调用

连接池管理 - 减少数据库连接开销

多缓存后端 - File / Memcache / Redis / OpenResty

极速路由匹配 - $O(1)$ 时间复杂度

无状态设计 - 关闭 PHP Session，最大化性能

PDO 预处理 - 防止 SQL 注入

查询性能统计 - 识别慢查询

2. 核心特性

2.1 链式数据库操作

框架提供类似 ThinkPHP5 的链式调用语法：

```
DB::table('user')->where('status', 1)->select(); DB::table('user')->where('id', 1)->find(); DB::table('user')->insert(['name' => 'test']);
DB::table('user')->where('id', 1)->update(['name' => 'new']); DB::table('user')->where('id', 1)->delete();
```

2.2 性能优化措施

特性	说明
连接池管理	最大10连接，最小2连接，自动管理
查询缓存	支持多种缓存后端自动缓存
预处理语句	PDO 预处理，防止 SQL 注入
查询构建器	链式调用，延迟执行
性能统计	记录查询次数、执行时间，识别慢查询

2.3 路由匹配优势

与 inhere/sroute 融合，具备以下优势：

- 极速路由匹配 - O(1) 时间复杂度，不受路由数量影响
- 支持路由组 - 便于 API 版本管理
- 中间件支持 - 灵活的请求处理管道
- 命名路由 - 便于生成 URL
- 参数匹配 - 支持动态路径参数

3. 快速开始

3.1 安装

```
composer require quickapi/framework
```

3.2 配置数据库

```
use QuickAPI\Database\DB; DB::config([ 'driver' => 'mysql', 'host' => '127.0.0.1', 'port' => 3306, 'database' => 'quickapi',  
'username' => 'root', 'password' => '', ]);
```

3.3 配置缓存

```
use QuickAPI\Cache\Cache; Cache::config([ 'driver' => 'redis', 'redis' => [ 'host' => '127.0.0.1', 'port' => 6379, ], ]);
```

3.4 定义路由

```
use QuickAPI\Route\Route; use QuickAPI\Core\Response; Route::get('/users', function() { return  
Response::json(DB::table('user')->select()); }); Route::get('/user/{id}', function($id) { return Response::json(  
DB::table('user')->where('id', $id)->find() ); });
```

3.5 运行应用

```
use QuickAPI\Core\App; $app = new App(); $app->run();
```

4. 数据库操作

4.1 基础查询方法

方法	说明
select()	查询所有记录，返回数组
find()	查询单条记录，返回一维数组或 null
count()	统计记录数量
insert(\$data)	插入数据，返回自增 ID
update(\$data)	更新数据，返回影响行数
delete()	删除数据，返回影响行数

4.2 链式方法一览

方法	说明
table(\$table)	设置表名
field(\$fields)	指定查询字段（字符串或数组）
where(\$field, \$value)	WHERE 等值条件
whereOr(\$field, \$value)	OR WHERE 条件
whereIn(\$field, \$values)	WHERE IN 条件
whereNotIn(\$field, \$values)	WHERE NOT IN 条件
whereBetween(\$field, \$min, \$max)	WHERE BETWEEN 条件
whereLike(\$field, \$value)	WHERE LIKE 模糊匹配
whereNull(\$field)	WHERE IS NULL
whereNotNull(\$field)	WHERE IS NOT NULL
join(\$table, \$on, \$type)	JOIN 关联查询
leftJoin(\$table, \$on)	LEFT JOIN
rightJoin(\$table, \$on)	RIGHT JOIN
order(\$field, \$dir)	排序（ASC / DESC）
group(\$field)	分组
having(\$cond)	HAVING 条件
limit(\$n, \$offset)	限制条数
page(\$page, \$perPage)	分页查询

4.3 查询示例

```
# 查询所有用户 DB::table('users')->select(); # 条件查询 DB::table('users')->where('status', 1)->select(); # 链式调用
DB::table('users')->field('id, name, email')->where('status', 1)->order('created_at', 'desc')->limit(10)->select(); # JOIN 关联查询
DB::table('posts')->field('posts.*, users.name as author')->join('users', 'posts.user_id = users.id')->select(); # 分页查询 $result =
DB::table('users')->paginate(15, $page); # $result['data'] - 当前页数据 # $result['total'] - 总记录数 # $result['current_page'] -
当前页码 # $result['last_page'] - 最后一页 # $result['per_page'] - 每页条数
```

4.4 写入示例

```
# 插入单条 $id = DB::table('users')->insert(['name' => '张三', 'email' => 'zhangsan@test.com', 'password' =>
password_hash('123456', PASSWORD_DEFAULT, [])); # 批量插入 DB::table('users')->insertAll([['name' => 'A', 'email' =>
'a@test.com'], ['name' => 'B', 'email' => 'b@test.com'], []]); # 更新 DB::table('users')->where('id', 1)->update(['name' => '新名字']);
# 删除 DB::table('users')->where('id', 1)->delete();
```

4.5 聚合查询

```
DB::table('users')->count(); // 计数 DB::table('orders')->sum('amount'); // 求和 DB::table('users')->avg('score'); // 平均值
DB::table('products')->max('price'); // 最大值 DB::table('products')->min('price'); // 最小值
```

4.6 事务操作

```
# 方式一：回调事务 DB::transaction(function() { DB::table('users')->insert([...]); DB::table('logs')->insert([...]); }); # 方式二：手动事务
DB::beginTransaction(); try { DB::table('users')->insert([...]); DB::commit(); } catch (Exception $e) { DB::rollback(); }
```

4.7 原生 SQL

```
# 查询 $rows = DB::query('SELECT * FROM users WHERE status = ?', [1]); # 执行 $affected = DB::execute('DELETE FROM logs
WHERE created_at < ?', ['2024-01-01']);
```

5. 缓存系统

5.1 支持的缓存驱动

驱动	配置项	说明
file	path	文件缓存，无需额外服务
redis	host, port, password	高性能内存缓存
memcache	host, port	分布式内存缓存
openresty	host, port	OpenResty 共享内存缓存

5.2 配置示例

```
# Redis 缓存 Cache::config([ 'driver' => 'redis', 'redis' => [ 'host' => '127.0.0.1', 'port' => 6379, 'password' => null, 'database' => 0, 'prefix' => 'quickapi:', ], ]); # 文件缓存 Cache::config([ 'driver' => 'file', 'file' => [ 'path' => '/tmp/cache', ], ]);
```

5.3 基本操作

```
# 设置 / 获取 / 删除 Cache::set('key', 'value', 300); // 缓存 300 秒 Cache::get('key', 'default'); // 获取，不存在返回默认值
Cache::has('key'); // 检查是否存在 Cache::delete('key'); // 删除 Cache::forget('key'); // 删除（别名） Cache::clear(); //
清空所有缓存 # 记住模式（缓存穿透自动回源） $users = Cache::remember('user_list', function() { return
DB::table('users')->select(); }, 300); // 缓存 5 分钟 # 获取并删除 $value = Cache::pull('key'); # 原子递增 / 递减
Cache::increment('counter', 1); Cache::decrement('counter', 1); # 批量操作 Cache::setMultiple(['k1'=>'v1', 'k2'=>'v2'], 300);
Cache::getMultiple(['k1', 'k2']); Cache::deleteMultiple(['k1', 'k2']);
```

5.4 标签缓存

```
# 设置带标签的缓存 Cache::tag('users')->set('list', $data); Cache::tag('users')->set('count', $count); # 清空某标签下所有缓存
Cache::tag('users')->flush();
```

5.5 缓存锁

```
# 获取锁（10秒超时） if (Cache::lock('resource:1', 10)) { // 执行独占操作 Cache::unlock('resource:1'); }
```

6. 路由系统

6.1 HTTP 方法路由

```
Route::get($path, $handler); // GET Route::post($path, $handler); // POST Route::put($path, $handler); // PUT Route::patch($path, $handler); // PATCH Route::delete($path, $handler); // DELETE Route::options($path, $handler); // OPTIONS Route::any($path, $handler); // 任意方法 Route::match(['GET','POST'], $path, $handler); // 指定多个方法
```

6.2 路由组

```
# 带前缀的路由组 Route::group(['prefix' => '/api/v1'], function() { Route::get('/users', 'UserController@index'); Route::get('/users/{id}', 'UserController@show'); }); # 带中间件的路由组 Route::group(['middleware' => ['Auth', 'Cors']], function() { Route::get('/profile', 'ProfileController@show'); }); # 嵌套路由组 Route::group(['prefix' => '/api'], function() { Route::group(['prefix' => '/v1'], function() { Route::get('/users', 'UserController@index'); }); });
```

6.3 参数约束

```
# 数字约束 Route::get('/user/{id}', 'Controller@show')->whereNumber('id'); # 字母约束 Route::get('/post/{slug}', 'Controller@show')->whereAlpha('slug'); # 字母数字约束 Route::get('/tag/{name}', 'Controller@show')->whereAlphaNum('name'); # UUID 约束 Route::get('/order/{uuid}', 'Controller@show')->whereUuid('uuid'); # 自定义正则约束 Route::get('/file/{path}', 'Controller@show')->where('path', '[a-z0-9/]+');
```

6.4 资源路由

```
Route::resource('users', 'UserController'); # 自动生成以下路由: # GET /users index (列表) # GET /users/{id} show (详情) # POST /users store (创建) # PUT /users/{id} update (更新) # DELETE /users/{id} destroy (删除)
```

6.5 其他路由功能

```
# 命名路由 Route::get('/profile', 'Controller@show')->name('profile'); $url = Route::route('profile'); // 生成 URL # 重定向 Route::redirect('/old', '/new', 301); # 404 处理 Route::set404(function($path) { return Response::error('接口不存在', 404); });
```

7. 请求与响应

7.1 Request 请求方法

方法	说明
method()	获取 HTTP 方法 (GET / POST 等)
uri()	获取请求路径
url()	获取完整 URL
all()	获取所有参数 (GET + POST)
query(\$key)	获取 GET 参数
post(\$key)	获取 POST 参数
header(\$key)	获取请求头
json()	获取 JSON 请求体 (自动解析)
file(\$key)	获取上传文件
cookie(\$key)	获取 Cookie
ip()	获取客户端 IP
user()	获取当前认证用户
isAjax()	是否 AJAX 请求
isJson()	是否 JSON 请求

7.2 Response 响应方法

```
# JSON 响应 Response::json(['data' => $value]); Response::json($data, 201); // 指定状态码 # 成功 / 错误快捷方法
Response::success(['user' => $user]); Response::error('操作失败', 500); # 分页响应 Response::paginate($data, $total, $page,
$perPage); # 其他类型 Response::text('纯文本'); Response::html('<h1>HTML</h1>'); Response::redirect('/new-url', 301);
Response::noContent(); // 204 # 链式调用 Response::json($data) ->status(201) ->header(['X-Custom' => 'value']) ->send();
```

8. 中间件

8.1 内置中间件

中间件	类名	说明
CORS	CorsMiddleware	跨域资源共享处理
Auth	AuthMiddleware	Token 用户认证
Throttle	ThrottleMiddleware	请求频率限流
Log	LogMiddleware	请求日志记录
Validate	ValidateMiddleware	请求数据验证

8.2 使用方式

```
# 全局中间件（在入口文件配置） $app->middleware(['Cors', 'Log']); # 路由组中间件 Route::group(['middleware' => ['Auth']],
function() { Route::get('/profile', 'ProfileController@show'); }); # 单个路由中间件 Route::get('/admin', 'AdminController@index')
->middleware(['Auth', 'Throttle']);
```

8.3 自定义中间件

```
class MyMiddleware { public function handle($request, callable $next) { // 前置逻辑 $response = $next($request); // 后置逻辑
return $response; } } # 注册 Middleware::register('my', MyMiddleware::class);
```

9. 验证器

9.1 内置验证规则

参数	说明
-	必填
-	合法邮箱
n	最小长度或最小值
n	最大长度或最大值
-	数字
-	整数
-	字符串
-	数组
v1,v2,...	值在列表中
v1,v2,...	值不在列表中
min,max	在范围内
pattern	匹配正则
-	合法 URL
-	合法 IP
-	仅字母
-	字母和数字
format	合法日期（默认 Y-m-d）
-	与字段_confirmation 匹配

9.2 使用示例

```
$validator = new Validator($data, [ 'name' => 'required|string|min:2|max:50', 'email' => 'required|email', 'password' => 'required|string|min:6|confirmed', 'age' => 'numeric|between:18,100', ]); if ($validator->fails()) { return Response::json([ 'error' => '验证失败', 'messages' => $validator->errors(), ], 422); } # 获取验证后的安全数据 $validated = $validator->validated(); # 获取第一条错误 $firstError = $validator->firstError();
```

10. 性能优化

10.1 连接池配置

```
'database' => [ // ... 其他配置 'pool' => [ 'max' => 10, // 最大连接数 'min' => 2, // 最小保持连接数 'idle_timeout' => 60, // 空闲超时 (秒) 'connect_timeout' => 5, // 连接超时 (秒) ], ],
```

10.2 查询缓存

```
# 使用 Cache::remember 自动缓存查询结果 $users = Cache::remember('active_users', function() { return DB::table('users')
->where('status', 1) ->order('created_at', 'desc') ->limit(50) ->select(); }, 300); // 缓存 5 分钟
```

10.3 性能统计

```
# 获取完整统计 $stats = DB::getQueryStats(); # 单独获取 DB::getQueryCount(); // 查询次数 DB::getTotalQueryTime(); // 总查询耗时 (毫秒) DB::getLastQueryTime(); // 最后一次查询耗时 DB::getSlowQueries(); // 慢查询列表 # 设置慢查询阈值 (毫秒) DB::setSlowQueryThreshold(100);
```

10.4 无状态设计

框架默认关闭 PHP Session，通过 Token 机制实现认证，避免 Session 读写带来的 I/O 开销，最大化并发性能。入口文件中已包含以下处理：

```
// 关闭已启动的 Session if (session_status() === PHP_SESSION_ACTIVE) { session_write_close(); } // 禁止自动启动
ini_set('session.auto_start', '0');
```

11. 配置参考

11.1 完整配置示例

```
return [ 'env' => 'development', 'debug' => true, 'timezone' => 'Asia/Shanghai', 'database' => [ 'driver' => 'mysql', 'host' => '127.0.0.1', 'port' => 3306, 'database' => 'quickapi', 'username' => 'root', 'password' => '', 'charset' => 'utf8mb4', 'prefix' => '', 'pool' => [ 'max' => 10, 'min' => 2, ], ], 'cache' => [ 'driver' => 'redis', 'redis' => [ 'host' => '127.0.0.1', 'port' => 6379, 'password' => null, 'database' => 0, 'prefix' => 'quickapi:', ], 'default_ttl' => 3600, ], 'middleware' => ['Cors', 'Log'], 'throttle' => [ 'max_attempts' => 60, 'decay_minutes' => 1, ], 'cors' => [ 'origin' => '*', 'methods' => 'GET, POST, PUT, DELETE, OPTIONS', 'headers' => 'Content-Type, Authorization', ], ];
```

附录 A 目录结构

```
quickapi/ +- src/ | +- Core/ # 核心类 (App, Request, Response) | +- Database/ # 数据库 (DB, Builder, Connection, Pool) | +-  
Cache/ # 缓存 (Cache, Drivers, QueryCache) | +- Route/ # 路由 (Route, RouteItem, Middleware) | +- Middleware/ # 内置中间件 | +-  
Validation/ # 验证器 | +- Exception/ # 异常类 +- demo/ # 完整演示项目 | +- index.php # 演示入口 | +- Controllers.php #  
控制器示例 | +- config.php # 配置文件 | +- init_database.php # 数据库初始化 | +- api_test.php # API 测试脚本 +- app/Controller/ #  
应用控制器 +- config/ # 配置文件 +- routes/ # 路由定义 +- tests/ # 单元测试 +- public/ # Web 入口 +- composer.json
```

附录 B 许可证

MIT License - 可自由使用、修改和分发。

(c) 2024 QuickAPI Framework. 保留所有权利。